

```

1  (*
2                                     CS51 Lab 7
3                               Modules and Abstract Data Types
4  *)
5
6  (*
7                                     SOLUTION
8  *)
9
10 (* Objective: This lab practices concepts of modules, including files
11    as modules, signatures, and polymorphic abstract data types.
12
13    There are three total parts to this lab. Please refer to the following
14    files to complete all exercises:
15
16        lab7_part1.ml -- Part 1: Implementing modules
17        lab7_part2.ml -- Part 2: Files as modules
18    -> lab7_part3.ml -- Part 3: Polymorphic abstract types and
19                                   Interfaces as abstraction barriers (this file)
20 *)
21
22 (*=====
23 Part 3: Polymorphic abstract types and
24     interfaces as abstraction barriers
25
26    It may feel at first that it's redundant to have both signatures and
27    implementations for modules. However, as we saw at the end of Part 2,
28    the use of signatures allows defining a set of functionality that
29    might be shared between two related (but still different) modules.
30
31    Another advantage of specifying signatures is that it provides a way
32    to hide, within a single module, various helper functions or
33    implementation details that we (as the developer of a module) would
34    rather not expose. In other words, it provides a rigid *abstraction
35    barrier* that when deployed properly makes it impossible for a
36    programmer using your module to violate desired invariants. This
37    abstraction barrier is a key tool toward satisfying the edicts of
38    prevention and compartmentalization.
39
40    Let's say we want to build a stack data structure. A stack is similar
41    to a queue (as described in Section 12.2 of the textbook) except that
42    the *last* element to be added to it is the *first* one to be
43    removed. That is, it operates as a LIFO (Last-In First-Out) structure.
44
45    To get started, we can implement stacks using lists. "Push" and "pop"
46    are commonly the names of the operations associated with adding an
47    element to the top or removing the topmost element, respectively. (The
48    terminology is reminiscent of a stack of plates in a cafeteria: you
49    begin with an empty pile, "push" one or more plates onto the top of
50    the stack, and at any point might "pop" a plate off the top of the
51    pile.)
52

```

```

53 In imperative programming languages, a "pop" function typically
54 *modifies* the stack by removing the topmost element from the stack as
55 a side effect and also returns the value of that element. However, in
56 the functional paradigm, side effects are eschewed. As a result, we'll
57 separate "pop" into two functions: "top", which returns the topmost
58 element from the stack (without removing it), and "pop", which returns
59 a new stack that has the topmost element removed.
60
61 We helped you out a little and defined `top` and `pop` for you,
62 below. These rely on a helper function called `pop_helper`, which
63 you'll need to implement. It should accept a stack and return a pair
64 containing the first element of the argument stack and a stack with
65 the first element removed. The `pop_helper` function isn't part of the
66 functionality of a stack implementation, just a help in implementing
67 that functionality.
68
69 You'll want to take advantage of the `EmptyStack` exception provided
70 in the module; raise it if an attempt is made to examine or pop the
71 top of an empty stack.
72
73 .....
74 Exercise 3A: Complete this implementation of a polymorphic stack module.
75 .....*)
76
77 module ListStack =
78   struct
79     exception EmptyStack
80
81     (* Stacks will be implemented as integer lists with the most
82        recently pushed elements at the front of the list. *)
83     type 'a stack = 'a list
84
85     (* empty -- An empty stack *)
86     let empty : 'a stack = []
87
88     (* push elt s -- Returns a stack like stack `stk` but with element
89        `elt` added to the top *)
90     let push (elt : 'a) (stk : 'a stack) : 'a stack =
91       elt :: stk
92
93     (* pop_helper stk -- Returns a pair of the top element of `stk`
94        and a stack containing the remaining elements after removing the
95        top element *)
96     let pop_helper (stk : 'a stack) : 'a * 'a stack =
97       match stk with
98       | [] -> raise EmptyStack
99       | hd :: tl -> (hd, tl)
100
101     (* top stk -- Returns the value of the topmost element on stack
102        `stk`, raising the `EmptyStack` exception if there is no
103        element to be returned. *)
104     let top (stk: 'a stack) : 'a =
105       fst (pop_helper stk)
106

```

```

107      (* pop stk -- Returns a stack with the topmost element from `stk`
108         removed, raising the `EmptyStack` exception if there is no
109         element to be removed. *)
110      let pop (stk : 'a stack) : 'a stack =
111          snd (pop_helper stk)
112      end ;;
113
114      (* Now let's use this implementation and consider some implications.
115
116      .....
117      Exercise 3B: Write a function `small_stack` that takes an argument of
118      type unit and uses the `ListStack` implementation to create and return
119      a new stack with the integer values `5` and then `1` pushed in that
120      order.
121      .....*)
122
123      let small_stack () : int list =
124          let open ListStack in
125              empty
126              |> push 5
127              |> push 1 ;;
128
129      (* .....
130      Exercise 3C: Now, use `ListStack` functions to write an expression
131      that defines `last_el` (not as `0` as below but instead) as the value
132      of the topmost element from `small_stack`.
133      .....*)
134
135      let last_el = ListStack.top (small_stack ()) ;;
136
137      (* Based on our requirements above, what should the value `last_el` be?
138
139      Look more closely at the type of `small_stack`: It's of type `unit ->
140      int list`. This is expected, since that's how we defined it, but this
141      also means that we have the ability to operate on the data structure
142      without using the provided *abstraction* specified by the module.
143
144      In other words, the `ListStack` module was intended to enforce the
145      invariant that the elements will *always* be pushed and popped in a
146      LIFO manner. But, we could altogether circumvent that merely by using
147      arbitrary list operations on the stack.
148
149      .....
150      Exercise 3D: Write a function that, given a stack, returns a stack
151      with the elements in reverse order, *without using any of the
152      `ListStack` methods* (but feel free to use `List` library functions).
153      .....*)
154
155      let invert_stack : 'a ListStack.stack -> 'a ListStack.stack =
156          List.rev ;;
157
158      (* By the way, no need to "rewrap" the function, that is,
159
160          let invert_stack (s : ListStack.stack) : ListStack.stack =

```

```

161     List.rev s ;;
162 *)
163
164 (* Now what would be the result of the `top` operation on a stack
165    inverted with `invert_stack`? Let's try it.
166
167 .....
168 Exercise 3E: Write an expression using `ListStack` methods to get
169 the top value from a `small_stack` inverted with `invert_stack` and
170 name the result `bad_el`.
171 .....*)
172
173 let bad_el = ListStack.top (invert_stack (small_stack ())) ;;
174
175 (* This is bad. We have broken through the *abstraction barrier*
176    defined by the `ListStack` module, by extracting an element from the
177    stack that was not the "last in". You may wonder: "if I know that the
178    module is defined by a list, why not have the power to update it
179    manually?"
180
181 Two reasons:
182
183 1. As we've just done, it was entirely possible for a user of the
184    module to completely modify a value in its internal representation.
185    Imagine what would happen for a more complex module that allowed us
186    to break an invariant! From Problem Set 3, what would break if a
187    person could change a bignum representing zero to directly set the
188    negative flag? Or construct a list of coefficients some of which
189    are larger than the base?
190
191 2. What if the developer of the module wants to change the module
192    implementation for a more sophisticated version? Stacks are fairly
193    simple, yet there are multiple reasonable ways of implementing the
194    stack regimen. And more complex data structures could certainly
195    need several iterations of implementation before settling on a
196    final version.
197
198 Let's preserve the abstraction barrier by writing an interface for this
199 module. The signature will manifest an *abstract* data type for stacks,
200 without revealing the *concrete* implementation type. Because of that
201 abstraction barrier, users of modules satisfying the signature will have
202 *no access* to the values of the type except through the abstraction
203 provided by the functions and values in the signature.
204
205 .....
206 Exercise 3F: Define an interface for a polymorphic stack, called
207 `STACK`, which should be structured as much as possible like the
208 implementation of `Stack` above. But be careful in deciding what type
209 of data your function types in the signature should use; it's not `a
210 list`, even though that's the type you used in your implementation.
211 .....*)
212
213 module type STACK =
214   sig

```

```

215     exception EmptyStack
216     type 'a stack
217     val empty : 'a stack
218     val push : 'a -> 'a stack -> 'a stack
219     val top : 'a stack -> 'a
220     val pop : 'a stack -> 'a stack
221 end ;;
222
223 (* Now, we'll restrict the `ListStack` module to enforce the
224 `STACK` interface to form a module called `SafeListStack`. *)
225
226 module SafeListStack = (ListStack : STACK) ;;
227
228 (* An aside: The idiomatic way to restrict a module to a
229 particular signature is to define it that way from the
230 start. That is, the definition of the `SafeListStack` module
231 (and the `ListStack` module for that matter) should have started
232
233     module SafeListStack : STACK = struct ... end
234
235 In this lab, we used this two-step restriction-after-the-fact
236 approach for pedagogical purposes, but you should in general
237 apply a signature to a module from the start. *)
238
239 (* .....
240 Exercise 3G: Let's redo Exercise 3B using the `SafeListStack`
241 abstract data type we've just built. Write a function `safe_stack`
242 that takes a unit and uses `SafeListStack` methods to return a
243 stack of type `SafeListStack.stack`, with the integers 5 and then 1
244 pushed onto it.
245
246 What type is `safe_stack`? You should no longer be able to
247 perform list operations directly on it, which means the stack
248 preserves its abstraction barrier.
249 .....*)
250
251 let safe_stack () =
252   let open SafeListStack in
253   empty
254   |> push 5
255   |> push 1 ;;
256
257 (* The type of `safe_stack` is
258
259     unit -> int SafeListStack.stack
260
261 Notice that the return type is the *abstract* type `int
262 SafeListStack.stack`, not `int list` or some other concrete type
263 whose values can be manipulated by functions others than those
264 allowed by the `STACK` interface.
265
266 For that reason even though our implementation of stacks implements
267 them as lists, you can't perform list operations (like `List.rev`)
268 on them. Try `List.rev (safe_stack ())` and see what happens. The

```

```

269 abstraction barrier that module signatures enforce makes that
270 impossible.
271
272     # List.rev (safe_stack ()) ;;
273             ^^^^^^^^^^^^^^^^^^^^^^^
274     Error: This expression has type int SafeListStack.stack
275            but an expression was expected of type 'a list
276 *)

```